



**Kampus
Merdeka**
INDONESIA JAYA

MODUL PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

S1-INFORMATIKA

Disusun Oleh :

TIM DOSEN

Praktikum 14

Database Connectivity

Tujuan Praktikum :

1. Mahasiswa mampu membuat kode Java berbasis OOP.
2. Mahasiswa mampu mengkoneksikan database ke aplikasi.

Alat dan Bahan :

1. IntelliJ Idea

Materi :

1. Database Connectivity

Dalam pengembangan aplikasi nyata, penyimpanan data secara persisten adalah suatu keharusan. Untuk itu, integrasi antara program Java dan database menjadi sangat penting. Pada bab ini, Anda akan mempelajari cara menghubungkan program Java berbasis OOP dengan database relasional menggunakan JDBC (Java Database Connectivity).

JDBC adalah API Java yang memungkinkan aplikasi untuk:

- a. Terhubung ke database
- b. Menjalankan perintah SQL
- c. Mengambil dan memodifikasi data

Komponen utama JDBC:

- a. DriverManager: Mengelola koneksi
- b. Connection: Representasi koneksi ke DB
- c. Statement / PreparedStatement: Menjalankan SQL
- d. ResultSet: Menyimpan hasil query SELECT

Langkah-Langkah Koneksi Java ke Database

Untuk menghubungkan Java ke database, ikuti langkah berikut:

- a. Siapkan database (contoh: MySQL)
- b. Tambahkan library JDBC driver (misalnya: mysql-connector-java.jar)



- c. Buat koneksi dari Java
- d. Kirim perintah SQL
- e. Kelola hasil dan tutup koneksi

Langkah Praktikum :

1. Buat database dengan MySQL sebagai berikut

```
MariaDB [(none)]> CREATE DATABASE db_pbo;  
[Query OK, 1 row affected (0.002 sec)]
```

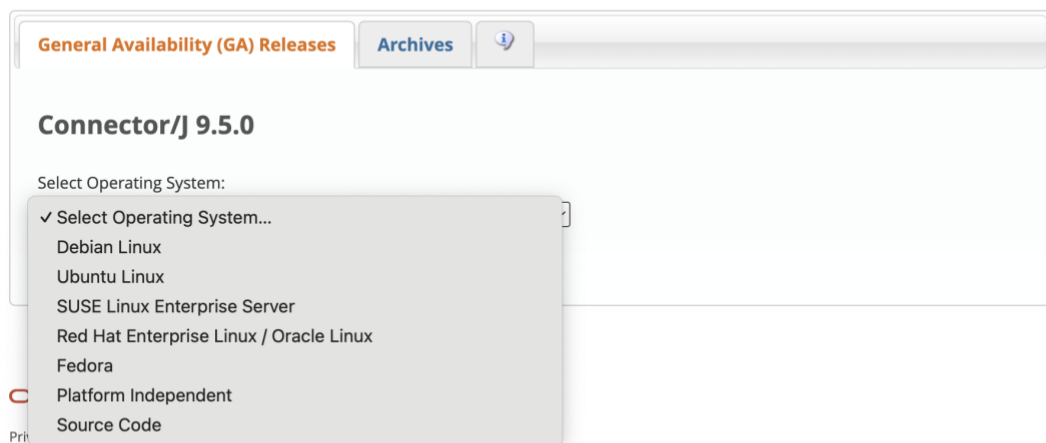
```
MariaDB [(none)]> USE db_pbo;  
[Database changed]
```

2. Download MySQLConnector pada link

<https://dev.mysql.com/downloads/connector/j/>

MySQL Community Downloads

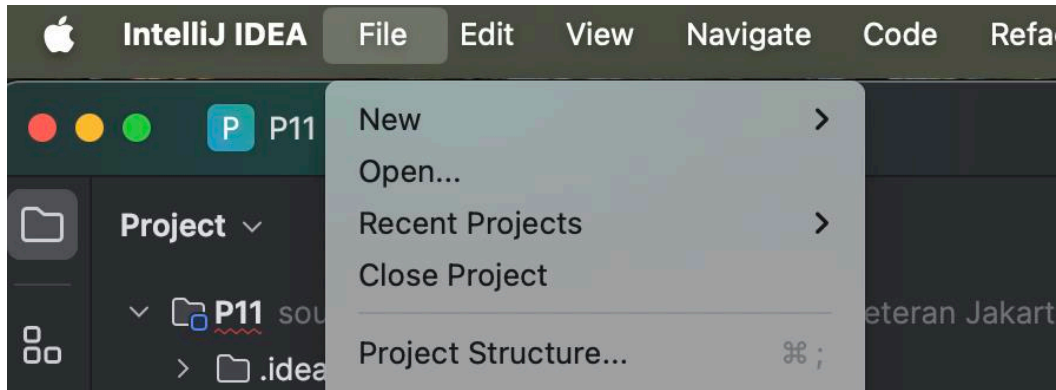
Connector/J



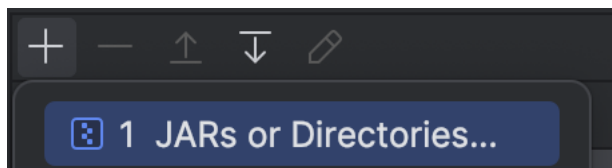
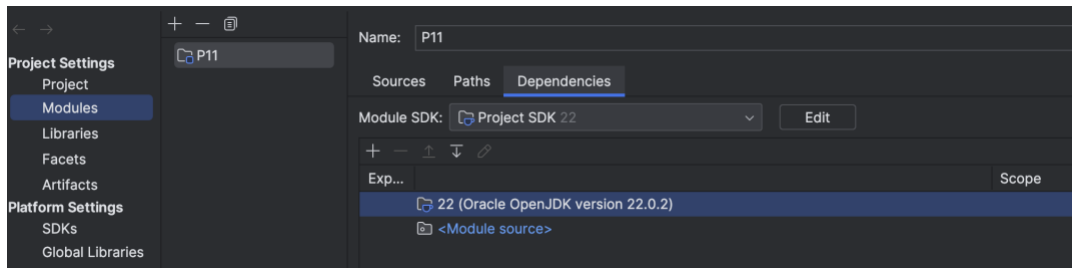
Pilih Platform independent untuk Windows/MacOS.



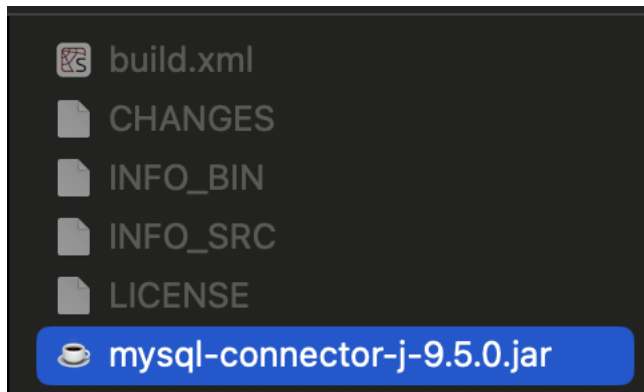
3. Config dan pada IntelliJ Idea dengan memilih project structure → modules → dependencies seperti gambar dibawah ini.



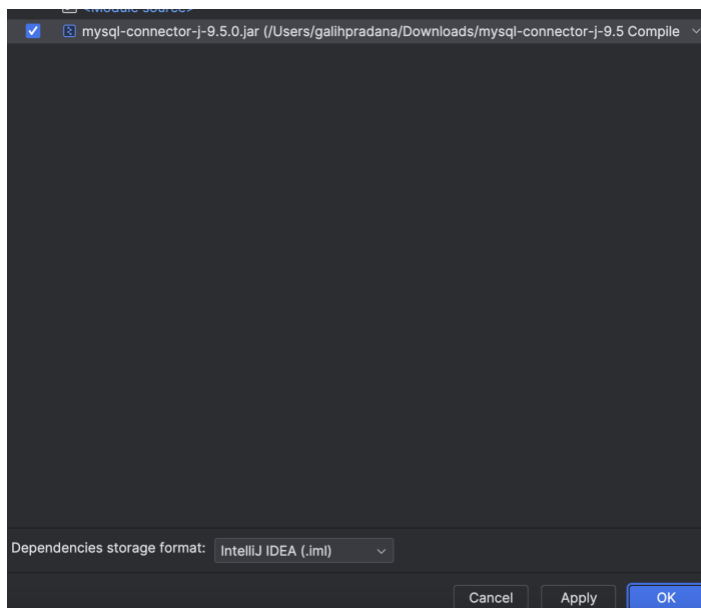
Klik tanda plus sebelah kiri lalu pilih JARs or Directories



Pilih mysql-connector yang sudah di download



Centang dan klik apply



4. Tes Koneksi

Buat kode berikut untuk mengecek koneksi dengan database

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Koneksi {
```

```

private static final String URL =
"jdbc:mysql://localhost:3306/db_pbo";
private static final String USER = "root";
private static final String PASSWORD = ""; // isi sesuai
konfigurasi Anda

public static Connection getConnection() throws SQLException
{
    return DriverManager.getConnection(URL, USER, PASSWORD);
}

public static void main(String[] args) {
    try (Connection conn = getConnection()) {
        if (conn != null) {
            System.out.println("Koneksi ke MySQL
berhasil!");
        }
    } catch (SQLException e) {
        System.out.println("Koneksi gagal: " +
e.getMessage());
    }
}
}

```

Jika berhasil maka akan muncul tampilan sebagai berikut

```
Koneksi ke MySQL berhasil!
```

```
Process finished with exit code 0
```

5. Setup Database

Setup database dengan membuat table pada database yang sudah dibuat sebelumnya.



```

import java.sql.Connection;
import java.sql.Statement;

public class SetupDatabase {
    public static void main(String[] args) {
        try (Connection conn = Koneksi.getConnection();
            Statement stmt = conn.createStatement()) {

            String sql = """
                CREATE TABLE IF NOT EXISTS mahasiswa (
                    nim VARCHAR(10) PRIMARY KEY,
                    nama VARCHAR(50),
                    jurusan VARCHAR(50)
                )
            """;
            stmt.execute(sql);
            System.out.println("Tabel mahasiswa berhasil
dibuat!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Maka hasilnya adalah sebagai berikut

```

Tabel mahasiswa berhasil dibuat!

Process finished with exit code 0

```



Kita cek pastikan hasil tablenya pada MySQL

```
[MariaDB [db_pbo]> show tables;
+-----+
| Tables_in_db_pbo |
+-----+
| mahasiswa        |
+-----+
1 row in set (0.002 sec)
```

Seharusnya tabel akan masih kosong dengan tampilan berikut

```
[MariaDB [db_pbo]> show tables;
Empty set (0.001 sec)
```

6. Insert Data

Selanjutkan masukan 10 data dengan kode berikut ini

```
import java.sql.Connection;
import java.sql.PreparedStatement;

public class InsertData {
    public static void main(String[] args) {
        String sql = "INSERT INTO mahasiswa (nim, nama,
jurusan) VALUES (?, ?, ?)";

        // Data 10 mahasiswa
        String[][] dataMahasiswa = {
            {"M001", "Andi Pratama", "Informatika"},
            {"M002", "Budi Santoso", "Sistem Informasi"},
            {"M003", "Citra Lestari", "Teknik Komputer"},
            {"M004", "Dewi Anggraini", "Informatika"},
            {"M005", "Eko Wijaya", "Teknik Komputer"},
```




```

        {"M006", "Fajar Nugroho", "Sistem Informasi"},
        {"M007", "Gita Permata", "Informatika"},
        {"M008", "Hendra Saputra", "Teknik Komputer"},
        {"M009", "Intan Kusuma", "Sistem Informasi"},
        {"M010", "Joko Prasetyo", "Informatika"}
    };

    try (Connection conn = Koneksi.getConnection();
        PreparedStatement pstmt =
conn.prepareStatement(sql)) {

        // Loop insert 10 data
        for (String[] mhs : dataMahasiswa) {
            pstmt.setString(1, mhs[0]); // nim
            pstmt.setString(2, mhs[1]); // nama
            pstmt.setString(3, mhs[2]); // jurusan
            pstmt.addBatch();
        }

        pstmt.executeBatch();

        System.out.println("10 data mahasiswa berhasil
dimasukkan!");
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```



Jika berhasil akan menampilkan output berikut ini

```
10 data mahasiswa berhasil dimasukkan!
```

```
Process finished with exit code 0
```

Cek pada database apakah sudah masuk data yang di insert

```
[MariaDB [db_pbo]> SELECT * FROM mahasiswa;
```

nim	nama	jurusan
M001	Andi Pratama	Informatika
M002	Budi Santoso	Sistem Informasi
M003	Citra Lestari	Teknik Komputer
M004	Dewi Anggraini	Informatika
M005	Eko Wijaya	Teknik Komputer
M006	Fajar Nugroho	Sistem Informasi
M007	Gita Permata	Informatika
M008	Hendra Saputra	Teknik Komputer
M009	Intan Kusuma	Sistem Informasi
M010	Joko Prasetyo	Informatika

```
10 rows in set (0.003 sec)
```

```
MariaDB [db_pbo]> █
```

7. Percobaan 1 : Menampilkan semua data

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;

public class LihatSemuaMahasiswa {
    public static void main(String[] args) {
        String sql = "SELECT * FROM mahasiswa";

        try (Connection conn = Koneksi.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {
```



```

        while (rs.next()) {
            System.out.println(
                rs.getString("nim") + " - " +
                rs.getString("nama") + " - " +
                rs.getString("jurusan")
            );
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

8. Percobaan 2 : Mencari mahasiswa berdasarkan jurusan

```

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class CariMahasiswaByJurusan {
    public static void main(String[] args) {
        String sql = "SELECT * FROM mahasiswa WHERE jurusan = ?";

        try (Connection conn = Koneksi.getConnection();
            PreparedStatement pstmt =
conn.prepareStatement(sql)) {

            pstmt.setString(1, "Informatika");

            ResultSet rs = pstmt.executeQuery();

```



```

        while (rs.next()) {
            System.out.println(
                rs.getString("nim") + " - " +
                rs.getString("nama") + " - " +
                rs.getString("jurusan")

            );
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

9. Percobaan 3 : Update data

```

import java.sql.Connection;
import java.sql.PreparedStatement;

public class UpdateMahasiswa {
    public static void main(String[] args) {
        String sql = "UPDATE mahasiswa SET nama = ? WHERE nim = ?";

        try (Connection conn = Koneksi.getConnection();
            PreparedStatement pstmt =
conn.prepareStatement(sql)) {

            pstmt.setString(1, "Citra Larasati");
            pstmt.setString(2, "M003");

```



```

        int affected = pstmt.executeUpdate();
        System.out.println("Data diupdate: " + affected);

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

10. Percobaan 4 : Hitung mahasiswa per jurusan

```

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class HitungMahasiswaPerJurusan {
    public static void main(String[] args) {
        String sql = "SELECT jurusan, COUNT(*) AS total FROM
mahasiswa GROUP BY jurusan";

        try (Connection conn = Koneksi.getConnection();
            PreparedStatement pstmt =
conn.prepareStatement(sql)) {

            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                System.out.println(rs.getString("jurusan") +
                    " = " + rs.getInt("total"));
            }

        } catch (Exception e) {

```



```
e.printStackTrace();  
    }  
}  
}
```

Lembar Kerja Praktik :

1. Buat program Java

Pada sebuah perguruan tinggi, proses pengambilan mata kuliah setiap awal semester dilakukan oleh mahasiswa untuk menentukan mata kuliah apa saja yang akan mereka pelajari dalam satu semester ke depan. Proses ini dikenal sebagai Kartu Rencana Studi (KRS). Dalam praktiknya, banyak kampus yang masih menggunakan proses semi-manual—mengisi formulir, menyerahkan ke bagian akademik, lalu menunggu validasi dosen pembimbing. Agar mahasiswa memahami integrasi Java dengan database dan mampu membuat aplikasi manajemen data secara profesional, maka dirancanglah studi kasus Sistem Manajemen KRS menggunakan Java OOP + JDBC. Studi kasus ini merupakan latihan dengan tingkat kompleksitas menengah, dirancang untuk menguji keterampilan mahasiswa dalam mengelola data secara langsung melalui program Java.

Sistem yang akan dibangun adalah aplikasi konsol sederhana yang mampu mengelola:

a. Data Mahasiswa

Data mahasiswa sudah dimiliki sejak praktikum sebelumnya, berisi: NIM, Nama, Jurusan. Data ini menjadi identitas utama mahasiswa dalam sistem.

b. Data Mata Kuliah

Daftar mata kuliah tersedia dalam tabel terpisah: Kode MK, Nama Mata Kuliah, Jumlah SKS. Tabel ini bersifat statis dan dapat berisi mata kuliah dasar yang umum ditawarkan pada semester awal.



- c. Data KRS. Tabel ini menghubungkan mahasiswa dengan mata kuliah yang dipilih. Struktur tabel ini mencerminkan relasi many-to-many, yaitu satu mahasiswa bisa mengambil banyak mata kuliah, dan satu mata kuliah bisa diambil oleh banyak mahasiswa.

Sistem ini menyediakan lima operasi utama:

Fitur 1 : Melihat Semua Mata Kuliah

Fitur 2 : Menambahkan Mata Kuliah ke KRS (Insert)

Fitur 3 : Menghapus Mata Kuliah dari KRS (Delete)

Fitur 4 : Menampilkan KRS Mahasiswa (Select + Join)

Fitur 5 : Menghitung Total SKS Mahasiswa

